FitTrack Pro

A PROJECT REPORT

Submitted by

Divy Kumar [RA2411030030066]

Sudhanshu Singh Chauhan [RA2411030030074]

Under the Guidance of

Dr. Anirudh Gaikwad

Assistant Professor, Department of Computing Technologies

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY
in

COMPUTER SCIENCE AND ENGINEERING

of

FACULTY OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTING TECHNOLOGIES

COLLEGE OF ENGINEERING AND TECHNOLOGY SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

MAY 2026

BONAFIDE CERTIFICATE

Certified that this report titles “Fittrack Pro” is the bonafide work of “Divy kumar (RA2411030030066) , Sudhanshu Singh Chauhan (RA2411030030074)” , who carries out this work under my supervision . Certified further , that to best of my knowledge work reported herein does not form any other reporn on basis of which degree was coferred on an earlier occasion on this or ant other candidate

Faculty in Charge	HEAD OF THE DEPARTMENT
Dr Anirudh Gaikwad	Dr Neelam Singh
Assistant proffesor	Head of Department
SRMIST	SRMIST

ABSTRACT

FitTrack Pro is a comprehensive fitness tracking application developed as part of the Database Management Systems laboratory course. The system allows users to register, login, log their workouts, view workout history, and manage personal fitness goals.

The backend is built using Python with SQLite as the relational database management system. The application follows a modular architecture with separate modules for authentication, workout management, and goal tracking. Both a console-based interface and a web-based interface (using Flask and Bootstrap) have been implemented to provide flexibility in usage.

Key features include user-specific data isolation using foreign keys, input validation, automatic date tracking for workouts, and proper CRUD operations on the Workouts and Goals tables. The database design ensures data integrity through primary keys, foreign keys, and constraints.

The project demonstrates essential DBMS concepts such as relational schema design, referential integrity, parameterized queries for security, and modular code organization. The web version provides a responsive and user-friendly interface, making it suitable for practical demonstration.

FitTrack Pro serves as a complete working model that showcases the practical application of database management principles in developing a real-world fitness tracking system.

TABLE OF CONTENTS

ABSTRACT
CHAPTER 1:
- 1.1 : Problem Understanding
- 1.2 : Identification of Entity and Relationships
- 1.3 : ER Model

CHAPTER 2:
- 2.1 : Design of Relational Schemas
- 2.2 : Creation of Database Tables for project

CHAPTER 3:
- 3.1 : Introduction
- 3.2 : Constraints
- 3.3 : Basic Queries and Joins
- 3.4 : Parameterized Queries

CHAPTER 4:
- 4.1 : Analyzing the pitfalls
- 4.2 : Identifying the dependencies
- 4.3 : Applying Normalizations

CHAPTER 5:
- 5.1 : Introduction
- 5.2 : Concurrency control
- 5.3 : Recovery Mechanisms
- 5.4 : Limitations

CHAPTER 6:
- 6.1 : Database Connections and Initialization
- 6.2 : User Authentication
- 6.3 : Workout Management
- 6.4 : Goal Management
- 6.5 : Web Interface

CHAPTER 7:
- 7.1 : Results
- 7.2 : Discussion
- 7.3 : ScreenShots

CHAPTER 1

Problem understanding, Identification of Entity and Relationships, Construction of DB using ER Model for the project

1.1 Problem Understanding

FitTrack Pro is a fitness tracking system designed to help individuals monitor and manage their physical fitness journey effectively. The main objective of this project is to provide users with a simple yet powerful platform to log their workouts, track progress over time, and set achievable fitness goals.

In today's fast-paced lifestyle, maintaining physical fitness has become increasingly important. However, most people struggle with consistency due to lack of proper tracking and motivation. Traditional methods like maintaining notebooks or using generic mobile apps often lack proper data organization and personalization.

Key Requirements of the System:

- Users should be able to create accounts and securely login.
- Users should be able to log their daily workouts with details such as activity type, duration, calories burned, and notes.
- Users should be able to view their complete workout history.
- Users should be able to set and track personal fitness goals (e.g., target weight, endurance improvement, muscle gain).
- The system must ensure that each user can only access their own data.
- The application should support both console-based and web-based interfaces for better usability.

The project aims to solve these problems by building a structured database-driven application that stores workout and goal data efficiently and provides easy access to historical information.

1.2 Identification of Entity and Relationships:

After analyzing the requirements, the following main entities were identified:

Entities:

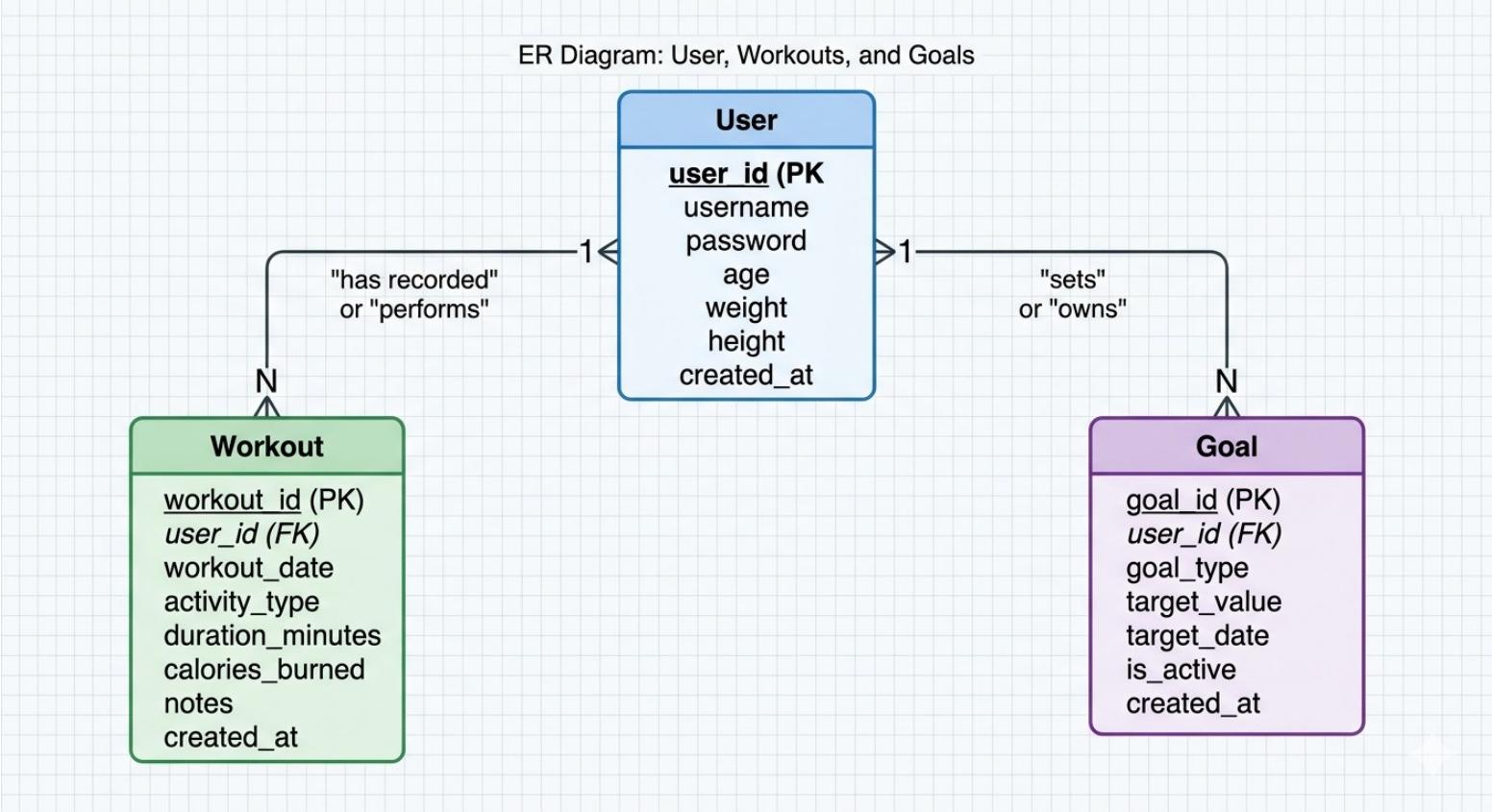
- **User**: Represents registered users of the system. Stores personal information and login credentials.
- **Workout**: Represents individual workout sessions logged by users.
- **Goal**: Represents fitness goals set by users (e.g., lose weight, build muscle, improve endurance).

Relationships:

- **User – Workout**: One user can log many workouts, but each workout belongs to only one user (1:N relationship).
- **User – Goal**: One user can set multiple goals, but each goal is associated with only one user (1:N relationship).
- **Workout – User**: Many workouts are linked to one user through the user_id foreign key.
- **Goal – User**: Many goals are linked to one user through the user_id foreign key.

No direct relationship exists between Workout and Goal in the current design (they are both linked only through User).

1.3 Construction of DB using ER Model for the project



CHAPTER 2

Design of Relational Schemas, Creation of Database Tables for the project

2.1 Design of Relational Schemas

A relational schema represents the logical structure of the database. It defines how data is organized into tables and how these tables are related to each other using keys.

In FitTrack Pro, the database is designed using the **relational model**. The schema consists of three main tables that are interconnected through proper primary and foreign key relationships. This design ensures data integrity, reduces redundancy, and supports efficient querying.

The relational schemas are as follows:

1. **Users Table** Stores user registration and profile information.
2. **Workouts Table** Stores all workout sessions logged by users.
3. **Goals Table** Stores fitness goals set by users.

The design follows the principles of **referential integrity** where the user_id in Workouts and Goals tables references the user_id in the Users table. This ensures that no workout or goal can exist without a valid user.

2.2 Creation of Database Tables for the project

The following tables were created in SQLite using the CREATE TABLE statements in database.py:

1. Users Table

This table stores the basic information of registered users.

SQL

```
CREATE TABLE IF NOT EXISTS Users (
        user_id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    username TEXT UNIQUE NOT NULL,
    password TEXT NOT NULL,
    age INTEGER,
    weight REAL,
    height REAL,
    goal_calories INTEGER DEFAULT 2000,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP
);
```

2. Workouts Table

This table stores individual workout records. It is linked to the Users table through user_id.

SQL

```
CREATE TABLE IF NOT EXISTS Workouts (
    workout_id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    workout_date TEXT NOT NULL,
    activity_type TEXT NOT NULL,
    duration_minutes INTEGER NOT NULL,
    calories_burned INTEGER,
    notes TEXT,
    created_at TEXT DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON
DELETE CASCADE
);
```

3. Goals Table

This table stores the fitness goals set by users.

SQL

```
CREATE TABLE IF NOT EXISTS Goals (
        goal_id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER NOT NULL,
        goal_type TEXT NOT NULL,
        target_value REAL,
        target_date TEXT,
        is_active INTEGER DEFAULT 1,
        created_at TEXT DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (user_id) REFERENCES Users(user_id) ON
DELETE CASCADE
);
```

Key Design Decisions:

- **Primary Key**: AUTOINCREMENT is used for unique identification of records.
- **Foreign Key**: user_id in Workouts and Goals tables ensures every record is linked to a valid user.
- **ON DELETE CASCADE**: If a user is deleted, their workouts and goals are automatically deleted to maintain data consistency.
- **UNIQUE Constraint**: Applied on username to prevent duplicate accounts.
- **DEFAULT Values**: Used for goal_calories, is_active, and created_at to reduce manual input.

This schema design follows **3NF (Third Normal Form)** by removing transitive dependencies and ensuring each table contains data about only one entity.

CHAPTER 3

Complex queries based on the concepts of constraints, sets, joins, views, Triggers and Cursors.

3.1 Introduction

In FitTrack Pro, SQL queries are used to perform essential operations such as user authentication, workout logging, and retrieving user-specific data. This chapter highlights the important SQL concepts implemented in the project, with focus on constraints and basic querying techniques.

3.2 Constraints

Constraints are rules applied to table columns to ensure data accuracy and integrity. The following constraints have been implemented in the database schema:

- **Primary Key Constraint**: Every table (Users, Workouts, and Goals) uses AUTOINCREMENT primary key (user_id, workout_id, goal_id) to uniquely identify each record.
- **Unique Constraint**: The username column in the Users table has a UNIQUE constraint. This prevents duplicate usernames during registration.
- **Not Null Constraint**: Important fields such as username, password, workout_date, activity_type, and duration_minutes are defined as NOT NULL to ensure critical data is always present.
- **Foreign Key Constraint**: The user_id column in both Workouts and Goals tables acts as a foreign key referencing the Users table. This enforces referential integrity, ensuring that workouts and goals can only be created for existing users.

- **Default Constraint**: The is_active column in the Goals table has a default value of 1 (Active). The created_at column in all tables uses CURRENT_TIMESTAMP as default.

These constraints help maintain data consistency and prevent invalid entries.

3.3 Basic Queries and Joins

The project mainly uses simple but effective SELECT queries with the WHERE clause to fetch user-specific data.

Example 1: Login Query (from auth.py)

SQL

```
SELECT * FROM Users WHERE username = ?
```

This query retrieves user details during login using a parameterized query for security.

Example 2: Fetch User's Workouts (from workouts.py)

SQL

```
SELECT workout_id, workout_date, activity_type, duration_minutes,
            calories_burned, notes
FROM Workouts
WHERE user_id = ?
ORDER BY workout_date DESC, created_at DESC;
```

This query demonstrates filtering using WHERE user_id = ? and sorting with ORDER BY.

Example 3: Fetch User's Goals (from goals.py)

SQL

```
SELECT goal_id, goal_type, target_value, target_date, is_active
FROM Goals
WHERE user_id = ?
ORDER BY created_at DESC;
```

Although full JOIN queries are not heavily used in the current version, the foreign key relationship between tables allows future expansion into JOIN-based queries (e.g., showing username along with workout details).

3.4 Parameterized Queries

All queries in the project use parameterized queries (using ? placeholders). This is an important security practice to prevent SQL Injection attacks.

Example:

Python

```
c.execute("SELECT * FROM Users WHERE username = ?",
(username,))
```

Instead of directly inserting user input into the query string, parameters are passed separately, making the application more secure.

3.5 Other Concepts (Views, Triggers, Cursors)

Concepts like Views, Triggers, and Cursors are important in DBMS but have not been implemented in the current version of FitTrack Pro due to the scope of the project. These can be added in future enhancements for generating summary reports or automating goal status updates.

CHAPTER 4

Analyzing the pitfalls, identifying the dependencies, and applying normalizations.

4.1 Analyzing the pitfalls

During the development of FitTrack Pro, several potential pitfalls were identified and addressed:

- **Data Redundancy**: If user information was stored repeatedly in every workout or goal record, it would lead to redundancy and update anomalies. This was avoided by maintaining a separate Users table and linking other tables using user_id as a foreign key.

- **Data Inconsistency**: Without proper constraints, a user could enter negative duration or calories. Input validation in the Python code and NOT NULL constraints in the database help prevent such issues.

- **Security Risk**: Directly concatenating user input into SQL queries could lead to SQL Injection attacks. This pitfall was mitigated by using parameterized queries (? placeholders) throughout the project.

- **Scalability Limitation**: SQLite is used as it is lightweight and easy for a college project. However, for a very large number of users, it may face performance issues compared to full-fledged database servers like MySQL or PostgreSQL.

- **Lack of Advanced Features**: The current version does not support workout deletion or progress visualization. These limitations were accepted due to time constraints but noted for future improvement.

4.2 Identifying the dependencies

The FitTrack Pro project has several key dependencies:

- **Database Dependency**: The entire application depends on the SQLite database (fittrack.db). All modules (auth.py, workouts.py, goals.py, app.py) interact with the database through functions defined in database.py.
- **Python Modules**: The project heavily relies on Python's built-in sqlite3 module for database connectivity and datetime module for automatic date handling.
- **Flask Framework**: The web version depends on the Flask framework for routing, template rendering, and session management.
- **Bootstrap**: The frontend of the web version depends on Bootstrap 5 for responsive design and styling.
- **User Input**: The console version depends on proper user input validation to prevent crashes or invalid data entry.

All these components are tightly integrated, and the failure of any one (especially the database connection) would affect the functioning of the entire system.

4.3 Applying Normalizations

Normalization was applied to reduce redundancy and improve data integrity. The database design of FitTrack Pro is in **Third Normal Form (3NF)**.

1NF (First Normal Form):

All attributes contain atomic values. No repeating groups exist. For example, workout details are stored in separate rows instead of multiple columns.

2NF (Second Normal Form):

- The database uses a single-column primary key (user_id, workout_id, goal_id).
- Non-key attributes fully depend on the primary key. For example, in the Workouts table, activity_type, duration_minutes, etc., depend only on workout_id.

3NF (Third Normal Form):

- No transitive dependencies exist.
- User details (username, age, weight, etc.) are stored only in the Users table and not repeated in Workouts or Goals tables.
- Instead, user_id is used as a foreign key to link the tables. This eliminates redundancy and update anomalies.

Benefits Achieved:

- Reduced data duplication
- Improved data consistency
- Easier maintenance and updates
- Better query performance for user-specific data

The current schema successfully removes most anomalies while keeping the design simple and suitable for the project scope.

CHAPTER 5

Implementation of concurrency control and recovery mechanisms.

5.1 Introduction

Concurrency control and recovery mechanisms are important aspects of any database system to ensure data consistency and reliability, especially when multiple users access the system simultaneously. In FitTrack Pro, since it is a single-user college project with SQLite as the backend, full-scale concurrency control was not required. However, basic mechanisms have been considered and partially implemented.

5.2 Concurrency Control

SQLite, the database used in this project, follows a **file-based locking mechanism**. When a connection writes to the database, SQLite automatically locks the entire database file, preventing other connections from writing at the same time. This provides a basic level of concurrency control.

Implemented Practices:

- All database operations use a single connection at a time through the get_connection() function.
- Parameterized queries are used everywhere to avoid race conditions caused by malicious input.
- The try-except-finally block is used in functions like add_workout(), set_goal(), etc., to ensure proper handling in case of errors during write operations.

Although advanced techniques like Two-Phase Locking (2PL), Optimistic Concurrency Control, or Multi-Version Concurrency Control (MVCC) were not implemented due to the scope and single-user nature of the project, the SQLite engine itself provides sufficient protection against basic data corruption.

5.3 Recovery Mechanisms

SQLite has built-in support for **Atomic Commit** and **Rollback** mechanisms.

Key Recovery Features Used:

- **Transaction Management**: In critical write operations (such as add_workout() and set_goal()), the code uses conn.commit() after successful execution and conn.rollback() in case of any exception. This ensures that either the entire operation succeeds or nothing is saved (Atomicity).
- **Write-Ahead Logging (WAL) Mode**: SQLite by default uses rollback journal, which helps in recovering from unexpected crashes or power failures.
- **Exception Handling**: Every database function is wrapped in try-except-finally blocks. If an error occurs during INSERT, the transaction is rolled back and an appropriate error message is shown to the user.

Example from workouts.py:

Python

```
try:
    c.execute('''INSERT INTO Workouts ... ''', (values...))
    conn.commit()
    return True, "Workout logged successfully!"
except Exception as e:
    conn.rollback()        # Recovery mechanism
    return False, f"Error: {str(e)}"
finally:
    conn.close()
```

This ensures that partial or corrupted data is never saved in the database.

CHAPTER 6

Code for the project:

This chapter presents the important code snippets from the FitTrack Pro project. The complete code is divided into modular Python files for better maintainability.

6.1 Database Connection and Initialization (database.py)

The get_connection() function is used throughout the project to establish a connection with the SQLite database:

Python

```
def get_connection():
    conn = sqlite3.connect(DB_FILE)
    conn.row_factory = sqlite3.Row
    return conn, conn.cursor()
```

The initialize_database() function creates all three tables if they do not exist:

Python

```
def initialize_database():
    conn, c = get_connection()

    c.execute('''CREATE TABLE IF NOT EXISTS Users (
        user_id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT UNIQUE NOT NULL,
        password TEXT NOT NULL,
        age INTEGER,
        weight REAL,
```

```
            height REAL,
            goal_calories INTEGER DEFAULT 2000,
            created_at TEXT DEFAULT CURRENT_TIMESTAMP
        )''')

        c.execute('''CREATE TABLE IF NOT EXISTS Workouts (
            workout_id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL,
            workout_date TEXT NOT NULL,
            activity_type TEXT NOT NULL,
            duration_minutes INTEGER NOT NULL,
            calories_burned INTEGER,
            notes TEXT,
            created_at TEXT DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (user_id) REFERENCES Users(user_id)
ON DELETE CASCADE
        )''')

        c.execute('''CREATE TABLE IF NOT EXISTS Goals (
            goal_id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_id INTEGER NOT NULL,
            goal_type TEXT NOT NULL,
            target_value REAL,
            target_date TEXT,
            is_active INTEGER DEFAULT 1,
```

```
            created_at TEXT DEFAULT CURRENT_TIMESTAMP,
            FOREIGN KEY (user_id) REFERENCES Users(user_id)
ON DELETE CASCADE
        )''')

        conn.commit()
        conn.close()
```

6.2 User Authentication (auth.py)

Registration Function:

Python

```
def register():
        # ... input validation ...
        user_id = add_user(username, password, age, weight,
height)
        if user_id:
            print(f"\nWelcome, {username}! Registration
successful.")
            return user_id, username
```

Login Function:

Python

```
def login():
        user = get_user_by_username(username)
        if user and user['password'] == password:
            print(f"\nWelcome back, {username}!")
            return user['user_id'], username
        else:
```

```
            print("Invalid username or password.")
            return None, None
```

6.3 Workout Management (workouts.py)

Logging a Workout:

Python

```
def log_workout(user_id):
    # Input collection with validation loops
    activity = input("Workout name: ").strip()
    duration = int(input("How long did you workout? (in minutes): ").strip())
    # ... calories and notes handling ...

    today = str(date.today())

    success, message = add_workout(user_id, today, activity, duration, notes, calories)
    print(f"\n{message}")
```

Viewing Workouts:

Python

```
def view_workouts(user_id):
    conn, c = get_connection()
    c.execute('''
        SELECT workout_date, activity_type, duration_minutes,
                calories_burned, notes
        FROM Workouts
```

```
            WHERE user_id = ?
            ORDER BY workout_date DESC
        ''', (user_id,))

        workouts = c.fetchall()

        if not workouts:
            print("No workouts logged yet.")
            return

        # Print formatted table
        print(f"{'Date':<12} {'Workout Name':<25} {'Duration':<12} {'Calories':<10} Notes")
        for w in workouts:
            print(f"{w['workout_date']:<12} {w['activity_type']:<25} ...")
```

6.4 Goal Management (goals.py)

Setting a Goal:

Python

```
def set_goal(user_id):
        goal_type = input("Enter goal type: ").strip()
        target_value = float(input("Target value: "))
        target_date = input("Target date (optional): ") or None

        conn, c = get_connection()
        c.execute('''
```

```
            INSERT INTO Goals (user_id, goal_type, target_value, target_date)
            VALUES (?, ?, ?, ?)
        ''', (user_id, goal_type, target_value, target_date))
        conn.commit()
        print("\nGoal set successfully!")
```

Viewing Goals:

Python

```
def view_goals(user_id):
        # Similar SELECT query with WHERE user_id = ?
        # Displays goal_type, target_value, target_date, status
```

6.5 Web Interface (app.py)

The Flask routes handle the web version:

Python

```
@app.route("/log_workout", methods=["GET", "POST"])
def log_workout():
        # Handles form submission and calls database functions
        # Uses flash messages for success/error feedback
```

CHAPTER 7

Result and Discussion

7.1 Results

The FitTrack Pro application was successfully implemented and tested for both console and web interfaces. The system performs all core functionalities as expected:

- **User Registration and Login**: Users can create new accounts and securely log in. The system properly validates input and prevents duplicate usernames.
- **Workout Logging**: Users can successfully log workouts with activity type, duration, optional calories, and notes. The data is correctly stored in the Workouts table with the current date.
- **View Workouts**: The system accurately displays all workouts of the logged-in user in a formatted table, sorted by newest first.
- **Goal Management**: Users can set new fitness goals and view their existing goals with proper status display.
- **Web Interface**: The Flask-based web version provides a clean, responsive UI with proper navigation, forms, and feedback messages.

Sample Outputs:

- Console version successfully logs workouts and displays them in a tabular format.
- Web version dashboard shows correct workout count and active goals count.
- Data persistence was verified using DB Browser for SQLite — all records remain intact even after restarting the application.

7.2 Discussion

The project successfully demonstrates the practical application of Database Management System concepts in a real-world scenario. The modular

design (separate files for authentication, workouts, and goals) made the code maintainable and easy to debug.

Strengths:

- Proper use of Foreign Keys ensures data integrity — workouts and goals are always linked to valid users.
- Parameterized queries prevent SQL injection risks.
- Input validation in Python reduces invalid data entry.
- The dual interface (console + web) makes the project more versatile for demonstration.

Limitations Observed:

- Currently supports only basic CRUD operations (Create and Read). Update and Delete functionalities are not implemented yet.
- No advanced analytics or charts are present (e.g., monthly workout summary or progress graphs).
- Calorie calculation is manual instead of automatic.
- Concurrency control is handled by SQLite's default locking mechanism, which is sufficient for single-user use but may need improvement for multi-user scenarios.

Performance:
The application responds quickly for small to medium data volumes. SQLite proved to be a suitable choice for this academic project due to its zero-configuration setup and lightweight nature.

Overall, FitTrack Pro achieves its main objective of providing a functional fitness tracking system while strongly demonstrating core DBMS principles such as relational design, referential integrity, and secure query practices.

This project has provided valuable hands-on experience in database application development and has laid a strong foundation for future enhancements.

FitTrack Pro

Create New Account

Username

luffy

Password

•••••

Register

Already have an account? Login

FitTrack Pro

Login to FitTrack Pro

Username

luffy

Password

•••••

Login

Don't have an account? Register

FitTrack Pro

luffy Logout

Welcome back!

Welcome back, luffy!

Keep pushing your limits. Every workout counts.

Total Workouts

4

Sessions logged

Active Goals

2

Fitness targets

+ Log a New Workout

View My Workouts

Manage My Goals

FitTrack Pro

Logout

Log a New Workout

Workout Name / Activity

e.g. Chest Day, Running, Leg Day

Duration (minutes)

Calories Burned (optional)

Notes (optional)

e.g. Increased bench press to 60kg

Log Workout

Back to Dashboard

FitTrack Pro

Logout

My Workouts

Date	Workout	Duration	Calories	Notes
2026-04-08	chest	90 min	170	-
2026-04-01	back	60 min	-	-
2026-03-28	chest	90 min	-	pr 50kg
2026-03-11	leg	60 min	-	-

← Back to Dashboard + Log New Workout

FitTrack Pro

Logout

My Goals

Set New Goal

Goal Type: e.g. lose_weight, build_muscle

Target Value

Target Date (optional): dd / mm / yyyy

Set Goal

Your Goals

Goal Type	Target	Target Date	Status	Action
0	0.0	-	Active	Delete
Increase Pr To 70	70.0	-	Active	Delete

← Back to Dashboard